

Helper Task Flow — How It Works

Adios · 8 October 2026 · Branch **feature/helper-flow** (from dev), not yet committed. Covers backend, customer app, driver app and admin.

At a glance

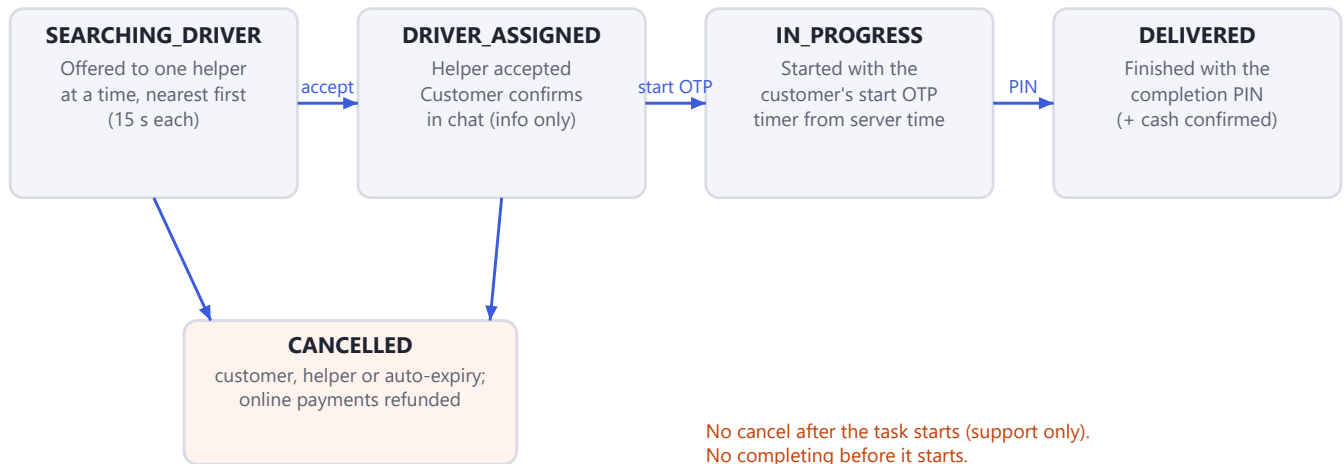
	Before	Now
Price	Calculated on the customer's phone; the server charged whatever it was sent (even ₹0).	Calculated on the server. The customer's offer must be 85–300% of the quoted fare.
Start	The start OTP was shown but never checked; helpers could skip straight to complete.	The helper must enter the customer's start OTP; the server checks it.
Finish	The completion PIN was sent to the helper's phone and compared there.	The helper never receives either code; the server checks the PIN.
Price raise (online)	Raised the price but collected no extra money.	The customer pays the difference through Razorpay; refunded with the task.
Nobody accepts	The task stayed "searching" forever.	Auto-cancelled (and refunded) 10 minutes after the search runs out.
Admin rates	Never applied (key mismatch HELPER vs helper).	New Helper pricing page; rates apply immediately.

Checks run

Project	Check	Result
backend	npm test (13 new unit tests), tsc	All pass · no type errors · lint 1 lower than dev
driver	typecheck, lint	Clean · lint unchanged from dev
app (customer)	typecheck, lint, jest	Same 5 pre-existing type errors (untouched files) · lint and payment tests pass
admin	build, lint, tsc	Builds · errors only in pre-existing, untouched files

Not yet done: an end-to-end run on real phones and Razorpay test mode. The step-by-step test plan is in section 9.

1. The task, start to finish



#	Who	What happens
1	Customer	Picks the task place (and optional drop-off), hours and description. The app shows the server's quote from GET /orders/helper-quote and a suggested range.
2	Customer	Sets an offer (-/+ limited to the allowed range), picks cash or online and taps Find a helper.
3	Server	Re-checks the offer against its own quote, creates the order (SEARCHING_DRIVER) and offers it to one helper at a time, nearest first, 15 seconds each. The offer shows the description and booked hours, never the codes.
4	Helper	Accepts → DRIVER_ASSIGNED. The helper lands on the task screen (chat is one tap away). The customer gets "Helper assigned" and sees the start OTP.
5	Customer	Confirms the task in chat (POST /orders/:id/confirm-assign). Saved on the order, so the helper sees it even if their chat was closed. Informational — it does not block Start.
6	Helper	Enters the customer's start OTP → IN_PROGRESS. The server stamps taskStartedAt and tells the customer's app, which moves to tracking.
7	Helper	Works; sends quick updates; the timer runs from the server's start time (survives app restarts) and shows overtime.
8	Helper	For cash: confirms the cash collected. Enters the completion PIN → DELIVERED. The invoice shows the real time worked.

If nobody accepts

- Each helper who passes or times out is counted; the customer sees "X of Y passed".
- When everyone has been offered it, the order is marked searchExhaustedAt and the customer sees "No helpers yet" with price-raise chips.
- Raising the price starts a fresh round: everyone nearby is offered it again, nearest first, with a new count.
- If the customer does nothing, the task is cancelled after **expiry minutes** (default 10) with the reason "No helper was available", and an online payment is refunded.
- A server restart no longer strands a search: the sweep (every minute) notices searches without a live dispatcher and treats them the same way.

2. Pricing

One formula, in backend/src/modules/pricing/helper.pricing.ts, used by the quote, cash orders, online checkout and price raises:

fare = round((base + hours × per-hour + max(0, km – free km) × per-km) × zone surge) + platform fee, then + tax %

Distance is the straight line from the task place to the drop-off (0 with no drop-off). The default rates are the ones the app always showed, so customers see the same prices as before.

Setting (admin → Helper pricing)	Default	Meaning
Base fare	₹40	Flat per task
Per hour	₹80	Per booked hour
Per km / free km	₹15 / 2 km	Distance to the drop-off beyond the free km
Platform fee	₹5	Flat, not multiplied by surge
Tax	5%	On the subtotal
Offer limits	85% – 300%	The customer's offer must be within this share of the fare
Booking hours	0.5 – 12	Hours outside this are clamped
Expiry	10 min	Auto-cancel after a search runs out

Worked examples

Task	Calculation	Fare	Offer range
2 hours, no drop-off	$40 + 160 + 0 = 200$; $+ 5 = 205$; $+ 5\% (10)$	₹215	₹183 – ₹645
1 hour, 5 km	$40 + 80 + 45 = 165$; $+ 5 = 170$; $+ 5\% (9)$	₹179	₹153 – ₹537
2 hours, 5 km (admin preview)	$40 + 160 + 45 = 245$; $+ 5 = 250$; $+ 5\% (13)$	₹263	₹224 – ₹789

3. Payments, price raises and refunds

Situation	What happens
Cash task, price raised	PATCH /orders/:id/increase-price. The cash the helper must collect goes up too.
Online task, price raised	The app opens Razorpay for the difference only (POST /payments/create-topup), then confirms through the usual /payments/verify. The raise applies only after Razorpay confirms the money. Applied exactly once even if the app, the callback and the webhook all report it.
Task taken while the customer is paying a raise	The raise can't apply, so that payment is refunded at once and the app says so.
Online task cancelled (by anyone, or auto-expiry)	The original payment and every raise payment are refunded through Razorpay.
Online task, old app calls increase-price	Refused with code TOPUP_REQUIRED; the new app switches to the top-up flow automatically.

Raise payments are tracked on their own Payment records (purpose "topup"); the order keeps a list of them. The admin Refunds page still shows only each order's first payment — raise refunds are visible in Razorpay and in the server log.

4. Rules the server now enforces

- **Status path** (helper orders only): searching → assigned → in progress → delivered. Cancel only before start. Staff can still cancel or complete a stuck task.
- **Start** needs the customer's start OTP and must come from the assigned helper.
- **Finish** needs the completion PIN and, for cash, the collected cash confirmed first.
- **Codes stay with the customer:** the start OTP and completion PIN are removed from everything sent to the helper (order details, accept, status updates, active job, new-job offer).

- **Price** from the client is never trusted: `customerPrice` in the request is ignored, and offers outside the range are refused.
- **Live updates** are checked against the order on screen, so one order's update can't move another order's screen.

5. What changed in each project

Backend

Area	Change
pricing/helper.pricing.ts (new)	The fare formula, rate defaults and limits; unit-tested.
orders/helper.flow.ts (new)	Status path, start/finish rules, hiding codes from helpers, helper push texts; unit-tested.
orders.service / controller / routes	Server pricing in <code>priceOrder</code> ; helper-quote and confirm-assign endpoints; start OTP check; price raise rebuilt (atomic, real re-dispatch, fresh round); expiry sweep; helper push texts.
payments (checkout, routes, refund)	create-topup endpoint, top-up settlement with a lease, top-up refunds and their webhooks.
models Order / Payment	assignConfirmedAt, taskStartedAt, taskCompletedAt, searchExhaustedAt, topupPayments; Payment purpose "topup".
dispatch.manager, socket.manager, index.ts	Records exhausted searches; stops the offer on cancel; saves chat confirmations; server sends <code>task_started</code> ; starts the expiry sweep; removed an unused socket event.
pricing.service, admin config	Rates key fix (HELPER/helper); helperRates read, validated and saved without wiping other settings.
invoice, errors	Invoice shows time worked; API errors can carry a code (e.g. TOPUP_REQUIRED).

Driver app

- Offer and task screen show the description, booked hours ("2 hrs", not "2 min") and both stops with directions.
- After accepting, the helper lands on the task screen. Start asks for the customer's start OTP; finish sends the PIN to the server; server error messages shown inline. No local code comparison.
- Timer runs from the server's start time and shows overtime. Customer's chat confirmation shown as information.
- 16 new strings in English, Telugu and Hindi.

Customer app

- Price from the server quote; offer buttons limited to the allowed range; "bidding" wording replaced by "Set your offer".
- Online price raise through Razorpay; clear messages for TOPUP_REFUNDED.
- Back while searching/assigned asks "Cancel this request?" — no more silently abandoned orders.
- Reopening an active task returns to the right step; Rebook pre-fills the form.
- Correct labels: Finding a helper / Helper assigned / Task in progress / Task completed; "From ₹80 / hour".
- About 40 new strings in English, Telugu and Hindi.

Admin

- New **Helper pricing** page (sidebar, after Zones): grouped fields, validation, live example fare, save with toasts. Translated.

6. Releasing

- **Ship the backend and the new driver app together.** Older driver builds start tasks without a code and will be refused.
- Older customer builds keep working: their prices are within the allowed range, and their chat confirmation is still saved.
- No data migration is needed; new fields default to empty.
- Set `PUBLIC_API_URL` so the Razorpay page can return to the app (already required for online payments).

7. Known limits and open questions

- **Invoice fees:** the invoice still uses a fixed ₹7/₹3 platform fee and an 18% GST split, while the app charges ₹5 + 5%. This is an accounting decision and was left unchanged — needs a decision.
- **Refunds page:** raise-payment refunds aren't listed in admin yet.
- **Server messages** (offer limits, 409s) are English only.
- **Chat messages and driver location** socket events carry no order id, so the app can't filter them yet.
- The helper's timer uses the phone clock against the server start time; a badly wrong phone clock skews the display only.

8. API summary

Endpoint	Who	Purpose
GET /orders/helper-quote	Customer	Fare, suggested range and offer limits for place, drop-off and hours
POST /orders/:id/confirm-assign	Customer	Confirm the task to the helper (saved)
PATCH /orders/:id/status	Helper	IN_PROGRESS with the start OTP; delivered with the PIN
PATCH /orders/:id/increase-price	Customer	Raise the price of a cash task
POST /payments/create-topup	Customer	Pay a raise on an online task (then /payments/verify)
GET / PUT /admin/config	Admin	Includes helperRates

New socket events: `task_started`, `assign_task_confirmed` (now sent by the server), `order_price_update`.

9. Test plan (condensed)

Two phones (customer + helper), backend with Razorpay test keys, admin open. Set expiry to 1 minute in admin while testing.

#	Test	Expected
1	npm test and tsc in backend	13 pass, no type errors
2	Admin → Helper pricing	Defaults shown; preview ₹263 (₹224–₹789); invalid values blocked; saves stick
3	Customer quote: 2 h, no drop-off	₹215; changes with hours and drop-off; –/+ stop at the limits
4	Helper offer	Shows "2 hrs booked", description, both stops; timeout passes it on; price raise re-offers with full details
5	Accept	Helper → task screen; customer → Helper assigned + start OTP + push
6	Chat confirm with helper chat closed	Helper sees "customer confirmed"
7	Start with wrong / right OTP	Wrong refused; right starts, customer moves to tracking; restart keeps the timer

#	Test	Expected
8	Finish: before cash / wrong PIN / right PIN	Blocked / refused / completed; invoice shows time worked
9	Online task + raise + cancel	Pays the difference only; cancel refunds both payments
10	Raise paid after a helper accepted	"Already taken" message; raise refunded
11	No helpers, wait past expiry	Auto-cancelled with "No helper was available"; online refunded
12	Cancel after start	Refused
13	Back while searching; reopen from Home / My Orders	Asks to cancel; reopens at the right step
14	Restart backend mid-search	Customer sees "No helpers yet" within ~2 minutes
15	Helper fetches the order via API	No deliveryOtp / restaurantPickupCode in the response
16	Ride, food and parcel orders	Work as before
17	Telugu and Hindi	No English or raw keys on the new screens